What is a local variable in Java ?

If we **declare a variable** inside the **method** OR **block** OR **constructor body** then It is called local OR Automatic OR temporary OR Stack Variable.

Example :

```
public void accept()
{
   int x = 100;    //Local Variable
}
```

As far as its scope is concerned, A local variable is **accessible** within the same method **only that means a local variable we cannot use outside of the method OR block OR constructor body.**

A local variable must be **initialized by developer** before use because we don't have default values for a local variable.

```
Example :
void main()
{
   int x;
   IO.println(x);  //Error, Initialization is compulsory before use, we don't have default value for LV
}
```

We cannot use/apply any **kind of modifier** (public, private, protected, static) on local variable **except final.**

```
Example :
----------
void accept()
{
   public int x = 100; //Invalid compilation error
   IO.println(x);
}
```

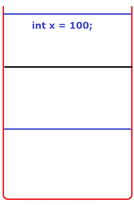
In Java, All the methods whether it is a static OR non static will be executed inside a very special memory called **STACK Memory**, which works on **LIFO** (Last In First Out) that means all the **local variables & parameter** variables are also executed as a part of **STACK** memory.

In Java, Whenever we call a method, a new **stack frame will be created for each & every method and all the local & parameter variables are executed as part of this particular Stack frame as shown in the program & diagram.**

```
MethodExecution.java
------------------------
void main()
{
    IO.println("Main method started");
    m1();
    IO.println("Main method ended");
}

void m1()
{
    IO.println("M1 method started");
    m2();
    IO.println("M1 method ended");
}

void m2()
{
    int x = 100;
    IO.println("I am m2 method");
    IO.println("Value of x is :"+x);
}
```




```
Main method started
M1 method started
I am m2 method
Value of x is :100
M1 method ended
Main method ended
```

Once the method execution is completed then automatically the **corresponding frame** will be deleted from the STACK memory and with that FRAME automatically local & parameter variables of that method Frame will also be deleted that is the reason local & parameter variables, we cannot use outside of the method.

* Each Stack Frame contains 3 parts :
  a) Local Variable Array
  b) Frame Data
  c) Operand Stack

Operators :

It is a symbol which describes that how a calculation will be performed on operands.
It performs specific operations on one, two, or three operands, and then return a result.

Types of Operators:

In general, three types of operators are available in Java : **Unary, binary and ternary**.

Operator Precedence

Operators	Precedence
postfix	`expr++ expr--`
unary	`++expr --expr +expr -expr ~ !`
multiplicative	`* / %`
additive	`+ -`
shift	`<< >> >>>`
relational	`< > <= >= instanceof`
equality	`== !=`
bitwise AND	`&`
bitwise exclusive OR	`^`
bitwise inclusive OR	`|`
logical AND	`&&`
logical OR	`||`
ternary	`? :`
assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`

Operator Precedence:

In order to calculate the result, we need to know Operator precedence. It describes operators are evaluated in what order.

Arithmetic Operator OR Binary Operator:

It is known as Arithmetic Operator OR Binary Operator because it works with minimum two operands.

Ex:- +, - , *, / and % (Modula Or Modulus Operator)

```
Optr1.java
------------
void main()
{
    int height = 3;
    int length = 5;

    var perimeter = 2 * height + 2 * length;
    IO.println(perimeter); //16

}
```

Note : The multiplication Operator has highest precedence than addition.

```
Optr2.java
-----------
void main()
{
    int height = 3;
    int length = 5;

    var perimeter = 2 * (height + 2) * length;
    IO.println(perimeter);   //50
}
```

Note: We can provide priority OR change the operator precedence by using () parentheses.
If two operators are same precedence than java will evaluate **left to right**.